

操作系统实践

D: 从零开始写操作系统

项目概况

- 项目背景：参考 uCore Tutorial，构建 OS 内核。
- 开发环境：Ubuntu 22.04 + QEMU 模拟器 + RISC-V GCC 工具链。
- 最终成果
 - 实现了 **LibOS** (裸机运行时环境)。
 - 实现了 **协作式多任务调度** (Multitasking)。
 - 实现了 **虚拟内存管理** (SV39 分页机制)。
 - 实现了 **进程管理** (fork 机制) 与 **交互式 Shell**。

LibOS 裸机运行时

文件结构：

```
os-project/
├── Makefile          <-- 指挥编译的脚本
└── os/
    ├── entry.S       <-- 1. 机器的入口（汇编）
    ├── kernel.ld     <-- 2. 内存排队（链接脚本）
    ├── sbi.c         <-- 3. 硬件接口
    ├── printf.c      <-- 4. 格式化输出工具
    └── main.c        <-- 5. 内核（主函数）
```

BatchOS 批处理系统

创建独立的用户程序 User App

将这个程序包进内核中

加载在 OS 运行时，把程序搬运到指定位置

执行移交控制权

没有权限直接操作串口硬件

用函数指针直接跳转，CPU 仍然处于高权限的 S-Mode

真正的 User App 应该运行在低权限的 U-Mode

Protection & Trap 特权级与陷入

实现 Trap 陷入

当 User App 想要打印字符时

发出 `ecall` 指令 → CPU 暂停 → 切换到内核 → 内核打印 → 切换回 App

```
os-project/  
├── Makefile  
└── os/  
    ├── entry.S  
    ├── kernel.ld  
    ├── sbi.c  
    ├── printf.c  
    ├── main.c  
    └── trap/  
        ├── trap.c  
        └── trap_entry.S
```

虚拟内存与页表

- 程序直接访问物理地址，比如：
 - 如果 Task A 想用地址 0x1000，Task B 也想用 0x1000，它们会冲突。
 - Shell 需要 fork，父子进程需要完全相同的内存布局，但数据又要隔离。
- RISC-V 的 SV39 分页机制
 - 建立一张映射表 Page Table
 - Task 访问相同的虚拟地址，但是通过硬件查表，映射到不同的物理地址。
- Paging.c
- 先在内存中建立页表并打印检查

独立的进程地址空间

- 目标
 - 给每个任务分配独立的页表
 - 实现用户空间映射：把User App 的代码映射到虚拟地址上，让 App 以为自己独占内存
 - 实现 satp 切换：当任务切换时，不仅切换栈(sp)，还要切换页表(satp)
- 修改 paging.c 扩充两个功能
 - Uvm_create: 创建一个空的用户页表
 - Uvm_map: 把物理内存映射到用户页表
- 任务结构体 task.c 给TCB增加一个成员pagetable，在创建任务时，给它分配独立的页表
- 修改user/linker.c 修改链接器，否则跳转地址对应不上

实现 fork

- 父进程中，返回子进程的 PID
- 子进程中，返回0
- 目标
 - 分配新的 TCB: 给子进程找位置
 - 复制地址空间: 遍历父进程的页表，将所有有效的数据页(代码、栈)都拷贝一份新的物理内存给子进程，并在子进程的页表中建立映射
 - 复制Trap上下文: 子进程需要从父进程fork的位置继续跑，所以寄存器状态要一致
- 实现
 - 修改paging.c 添加uvm_copy函数
 - 修改task.c 1. PID分配直接递增实现 2. 实现 task_fork 函数
 - 修改trap.c 注册系统调用 syscall 号 220

实现Shell

- 目标
 - 实现 readline 函数
 - 缓冲区: 定义一个 char buf[128] 存字符
 - 回显(Echo): 用户输入, 屏幕显示, 光标后移
 - 退格(Backspace): 内存中指针i减1, 屏幕上打印空格覆盖
 - 回车(Enter): 用户换行, 输入结束
- 修改 user/app.c 实现子进程和父进程的协作式调度

```
[ToyOS] Phase 6: Page Table Mapping
[Kernel] Checking BSS: recycled_ptr=0 (Expect 0)
[Kernel] Memory Manager Initialized.
[Kernel] Free RAM start: 8021c000, end: 88000000
[Kernel] stext=80200000, etext=80201000
[Kernel] Text Size=1000
[Kernel] Map UART... done.
[Kernel] Start mapping Text...
Mapping VA 80200000
Mapping VA 80201000
[Kernel] Map Text... done.
[Kernel] Map Rodata... done.
[Kernel] Map Data/BSS/Heap... done.
[Kernel] Paging ENABLED! Hello from Virtual World!
[Kernel] System matches Physical Memory 1:1.
[Kernel] Initializing tasks with Virtual Memory...
[Kernel] Task 0 created. PT=80260000
[Kernel] Idle -> Task 0

+++++
ToyOS Multitasking Shell v1.0
+++++
ToyOS > help
Commands:
  help - Show this message
  test  - Fork a child process to do work
  exit  - Shutdown OS
ToyOS > test
[Shell] Forking new process...
[Shell] Child created. I will toggle with child:
P [Child] I am a new process created by Shell!
  [Child] working: .P.P.P.P.
[Shell] Parent loop done.
ToyOS > exit
System Halt.
[Kernel] App 0 exited.
Done!
[Kernel] App 0 exited.
[Kernel] All tasks finished!
```